

Guía Clara: Cambios Locales y Deploy a Servidor (Frontend + Backend)

Objetivo: que siempre puedas hacer cambios en local, subirlos a GitHub y desplegarlos en producción sin romper nada.

Resumen rápido del flujo correcto:

1. Trabajas y pruebas en tu local.
2. Subes cambios a una rama de trabajo.
3. Fusionas a main solo cuando está validado.
4. En el servidor haces `git pull origin main`, build y recarga de servicios.

1) Ramas recomendadas (muy importante)

- `main`: producción (lo que vive en servidor).
- `develop` (opcional pero recomendado): integración/pruebas.
- `feature/*`: nuevas funcionalidades.
- `hotfix/*`: arreglos urgentes de producción.

Regla de oro: no trabajes directo en `main`.

Siempre crea una rama de trabajo y luego merge a `main`.

2) Flujo completo para FRONTEND

2.1 Trabajar en local

```
cd ~/ventas-app/frontend-ventas/frontend
git checkout main
git pull origin main
git checkout -b feature/mi-cambio-frontend

# haces tus cambios
npm install
npm run build
# opcional: npm run dev
```

2.2 Commit y push a GitHub

```
git status
git add .
git commit -m "feat(frontend): descripcion clara del cambio"
git push -u origin feature/mi-cambio-frontend
```

2.3 Merge a main

Puedes hacerlo por Pull Request (recomendado) o por consola:

```
git checkout main
git pull origin main
git merge --no-ff feature/mi-cambio-frontend
git push origin main
```

2.4 Deploy en servidor (frontend)

En tu VPS (ejemplo: /var/www/frontend):

```
cd /var/www/frontend
git checkout main
git pull origin main

# revisa variable de produccion (sin /api al final)
cat .env
# Debe ser: VITE_API_URL=http://prospectflow.it.com

npm ci
npm run build

# si nginx sirve desde /var/www/frontend/dist, solo recarga nginx
sudo systemctl reload nginx
```

2.5 Validación después de deploy (frontend)

1. Abre la web en incógnito o con hard refresh (Ctrl + F5).
2. En DevTools - Network, revisa que pegue a /api/auth/... y /api/prospects/....
3. Confirma que no aparezca localhost:3000 en requests.

3) Flujo completo para BACKEND

3.1 Trabajar en local

```
cd ~/ventas-app/backend-ventas
git checkout main
git pull origin main
git checkout -b feature/mi-cambio-backend
```

```
# haces cambios
npm install
npm run start    # o npm run dev
```

3.2 Commit y push

```
git status
git add .
git commit -m "fix(backend): descripcion clara del cambio"
git push -u origin feature/mi-cambio-backend
```

3.3 Merge a main

```
git checkout main
git pull origin main
git merge --no-ff feature/mi-cambio-backend
git push origin main
```

3.4 Deploy en servidor (backend)

```
cd /var/www/backend    # usa tu ruta real

git checkout main
git pull origin main

# valida variables de entorno
cat .env
# revisar DB_HOST, DB_NAME, DB_USER, DB_PASSWORD, JWT_SECRET

npm ci

# reinicia proceso segun como lo levantes
pm2 restart all
# o: pm2 restart nombre-del-proceso

# opcional logs
pm2 logs --lines 80
```

3.5 Validación después de deploy (backend)

```
curl -i http://prospectflow.it.com/health
curl -i http://prospectflow.it.com/api/auth/login \
  -H "Content-Type: application/json" \
  -d '{"email":"correo@dominio.com","password":"123456"}'
```

4) Variables de entorno: regla clara

- Frontend producción: VITE_API_URL=http://prospectflow.it.com (sin /api).
- Frontend local: VITE_API_URL=http://localhost:3000.
- Backend: nunca subas secretos reales a GitHub.

Importante en Vite: si cambias .env, debes volver a correr `npm run build` para que aplique en producción.

5) Checklist anti-errores (antes de subir y antes de deploy)

Antes de subir a GitHub

1. `git status` limpio o con cambios esperados.
2. Build local en verde.
3. No subir .env real ni secretos.
4. Mensaje de commit claro.

Antes de deploy en servidor

1. Confirmar que ya hiciste merge a main.
2. `git pull origin main` en servidor.
3. Variables correctas.
4. Build/restart según frontend o backend.

Después de deploy

1. Probar flujo real: login, listar prospectos, crear/editar.
2. Revisar Network (frontend) y logs (backend).
3. Si algo falla, rollback rápido al commit anterior.

6) Rollback rápido (si algo sale mal)

En servidor, puedes volver al commit anterior estable:

```
git log --oneline -n 5
# elegir commit anterior bueno

git checkout <commit_bueno>
# frontend: npm run build && sudo systemctl reload nginx
# backend: npm ci && pm2 restart all
```

Recomendación final: usa Pull Requests para todo cambio a main. Te ahorra errores y te da historial claro.

Documento generado para tu flujo actual de ProspectFlow (frontend + backend) con deploy en servidor Ubuntu y repos en GitHub.